Java Spring Boot MVC

Controladores, vistas Thymeleaf, servicios, formularios, validación y CRUD

UT03 · Primer bloque: MVC sin base de datos

MVC: separar responsabilidades

Modelo

Representa los datos y puede contener restricciones de validación.

Vista

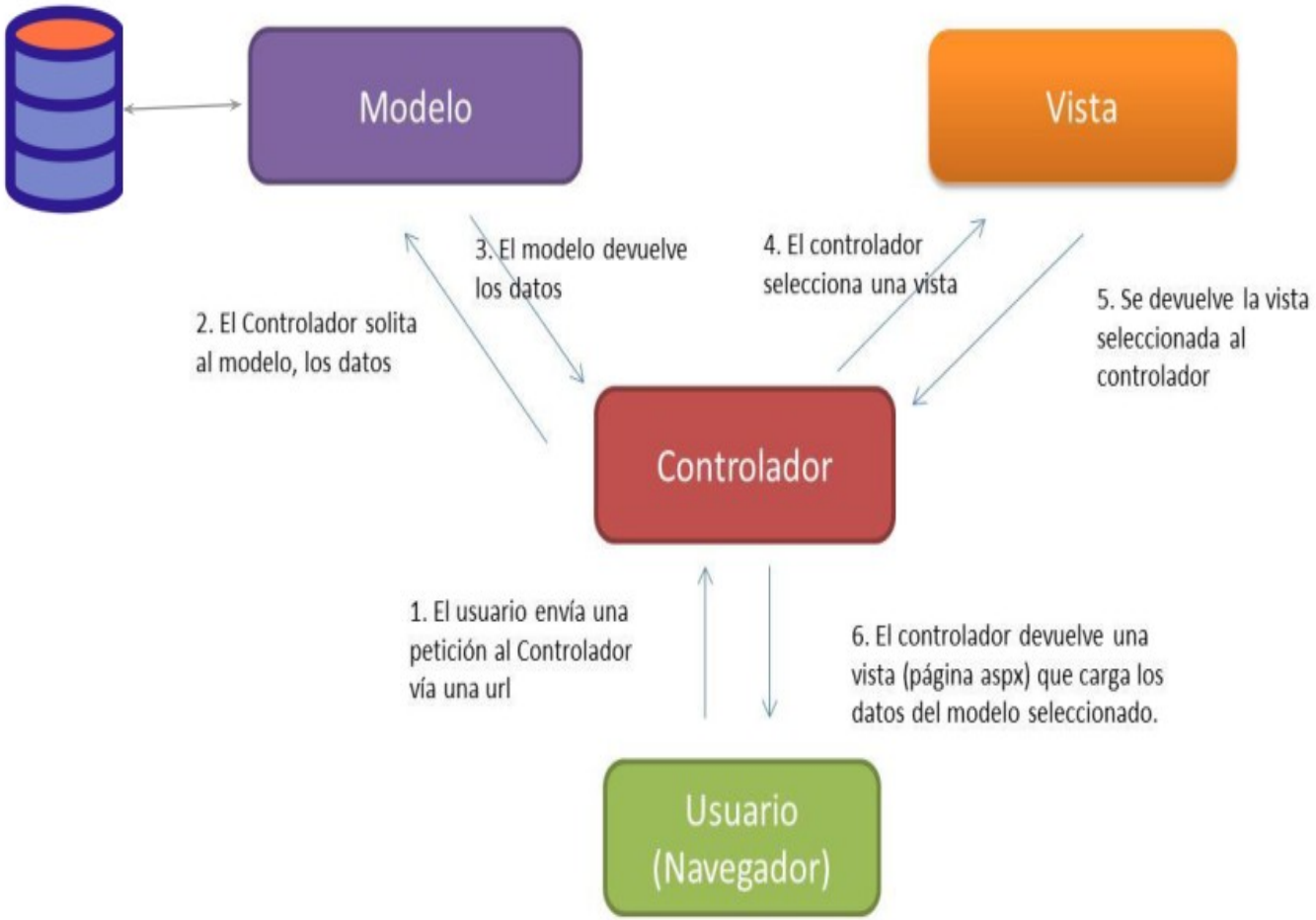
Genera el HTML que recibe el navegador. Usaremos Thymeleaf.

Controlador

Atiende las peticiones HTTP, invoca servicios y selecciona vistas.

Acceder a la base de datos corresponde a la capa de persistencia, no al modelo por sí solo.

MVC


Modelo
Vista
3. El modelo devuelve los datos
4. El controlador selecciona una vista
2. El Controlador solita al modelo, los datos
5. Se devuelve la vista seleccionada al controlador
Controlador
1. El usuario envía una petición al Controlador vía una url
6. El controlador devuelve una vista (página aspx) que carga los datos del modelo seleccionado.
Usuario (Navegador)

MVC

Modelos

- Son clases que representan los datos de la aplicación.
- Las clases de modelo usan lógica de validación para aplicar las reglas de negocio para esos datos.
- Normalmente, los objetos de modelo recuperan y almacenan el estado del modelo en una base de datos.
- Los datos actualizados se escriben en una base de datos.
- El modelo actualiza los datos en una base de datos, ya sea para crear, leer, actualizar o eliminar estos datos.
- Estas son las funciones básicas de la persistencia de información en bases de datos, a las que nos referiremos con el acrónimo CRUD (Create-Read-Update-Delete)

MVC

Vistas

- Son los componentes que muestran la interfaz de usuario (IU) de la aplicación.
- Por lo general, esta interfaz de usuario muestra los datos del modelo.
- Utilizaremos el motor de plantillas Thymeleaf para generar las páginas dinámicas con los datos del modelo.

MVC

Controladores

- Son clases que controlan las solicitudes que el usuario realiza a través del navegador.
- Recuperan datos del modelo.
- Llaman a plantillas de vista que devuelven una respuesta.

MVC y arquitectura por capas

El patrón MVC organiza la interacción web; los servicios y repositorios separan responsabilidades.

Presentación	Controlador + Thymeleaf
Negocio	Servicio: reglas y operaciones
Persistencia	Repositorio JPA (más adelante)

Ahora los datos vivirán en memoria; después conectaremos MySQL.

Terminología Java Utilizada

- Spring
- Spring Core
- Spring Boot
- Spring Boot Starter
- Spring Initializr
- Thymeleaf
- JDBC
- JPA
- Hibernate

Crear Nuevo Proyecto Spring

Utilizaremos la herramienta **Spring Initializr** para poder generar un proyecto **Spring Boot** de forma rápida.

Podremos utilizar Spring Initializr con la herramienta integrada en nuestro IDE (en eclipse → Spring Tools, o en IntelliJ Idea Community → Spring Initializr) o bien usar la versión web de Spring Initializr (https://start.spring.io/) para generar el paquete que descargaremos y abriremos con el IDE.

Crear un proyecto Spring Boot

Spring Initializr

Maven · Java · Java 21

Versión estable compatible de Spring Boot

Group y Artifact del proyecto.

Dependencias MVC

Spring Web MVC (Boot 4)

Thymeleaf

Para validaciones: Validation.

Elegiremos la misma versión de Spring Boot en todos los ordenadores del aula.

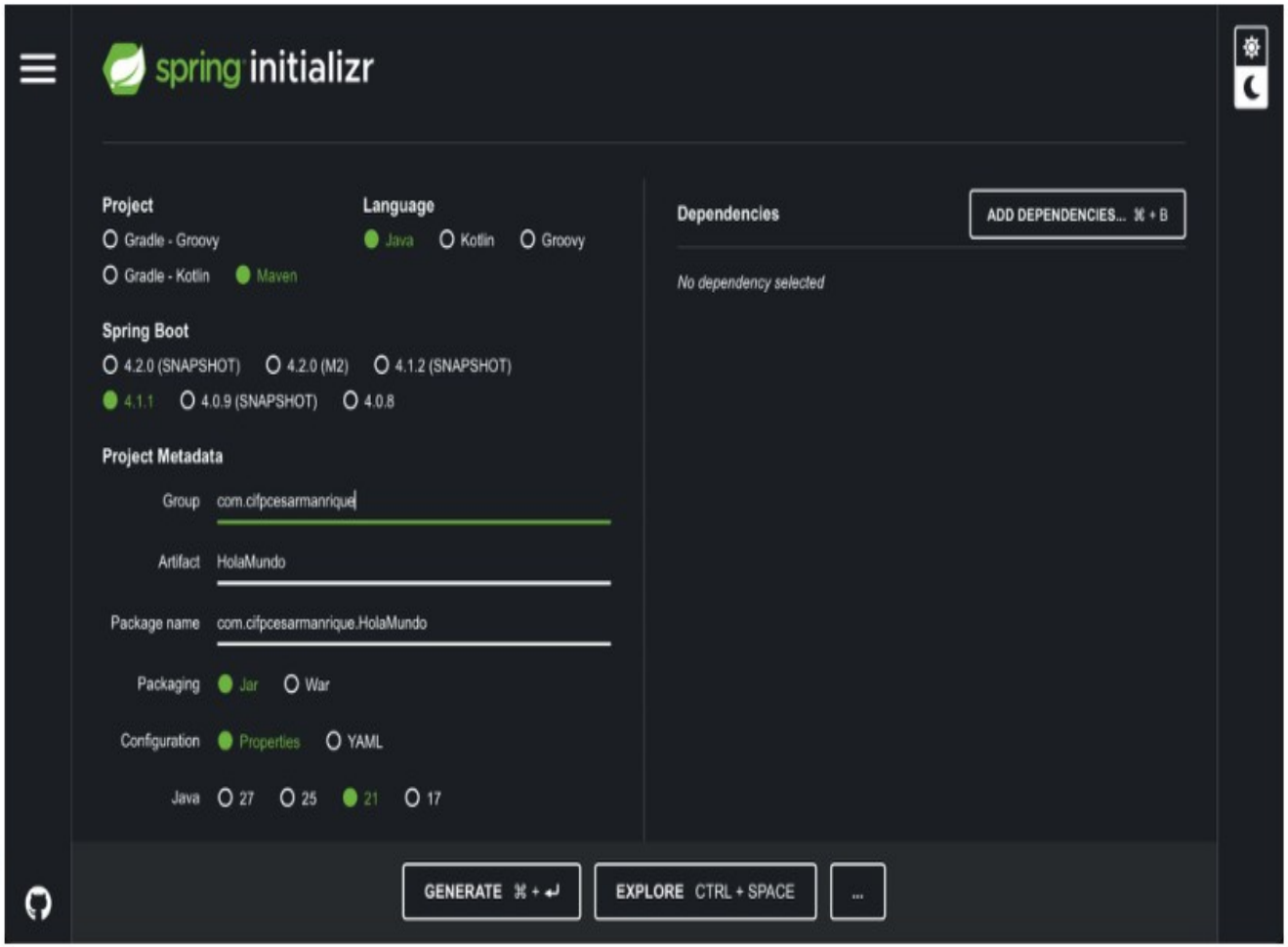
Crear Nuevo Proyecto Spring Web MVC

Para crear un proyecto web que soporte vistas, incluiremos las dependencias:

- Spring Web
- Thymeleaf

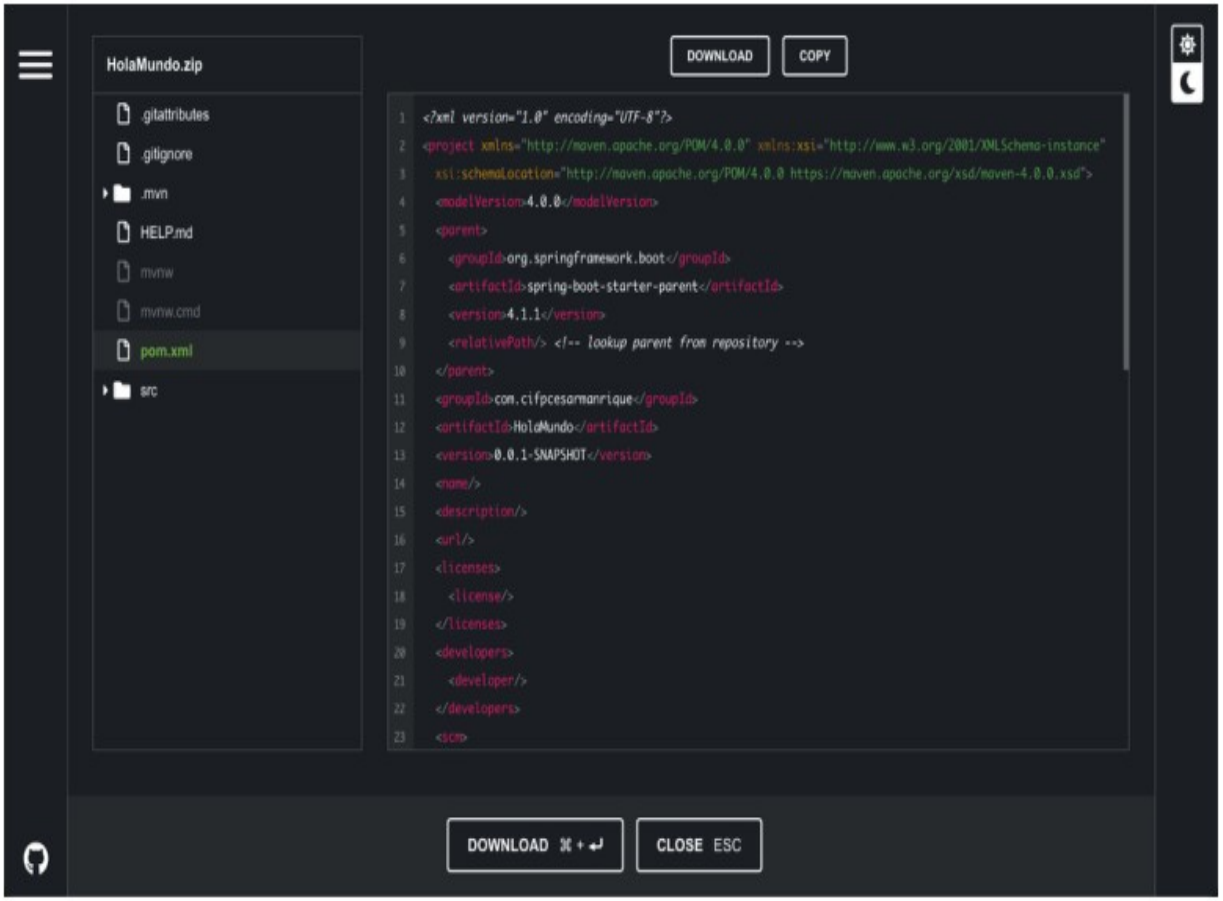
Generate → descargamos el archivo .zip que descomprimimos y abrimos con Intellij Idea

Explore → vemos el contenido del fichero pom.xml del proyecto



Crear Nuevo Proyecto Spring Web MVC

Si pulsamos EXPLORE vemos la estructura del proyecto genrado y el pom.xml con las dependencias añadidas.



Crear Nuevo Proyecto Spring Web MVC

Tras ejecutar el proyecto, nos muestra en consola la URL y puerto para abrirlo en el navegador, pero al no haber definido ningún *endpoint* mostrará error.

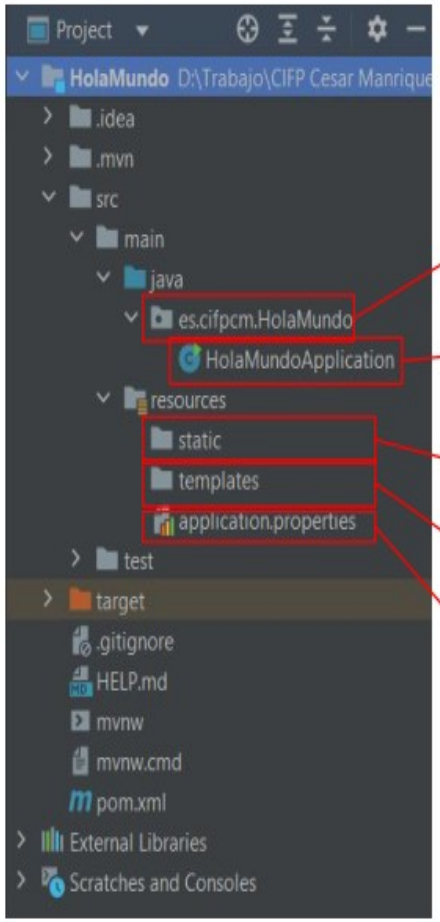
```
34Z  INFO 15156 --- [           main] e.cifpcm.HolaMundo.HolaMundoApplication  : No active profile set, falling back to 1 default profil
95Z  INFO 15156 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http)
96Z  INFO 15156 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
96Z  INFO 15156 --- [           main] o.apache.catalina.core.StandardEngine    : Starting Servlet engine: [Apache Tomcat/10.1.4]
81Z  INFO 15156 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
82Z  INFO 15156 --- [           main] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in
89Z  WARN 15156 --- [           main] ion$DefaultTemplateResolverConfiguration : Cannot find template location: classpath:/templates/ (p
```





Crear Nuevo Proyecto Spring Web MVC

Veamos los elementos importantes de la estructura básica de un proyecto generado:



Paquete principal del proyecto.
Contendrá subpaquetes para controladores, modelos, servicios, etc

Clase principal del proyecto identificada por la anotación **@SpringBootApplication** y que tiene el método **main**

Contendrá los elementos estáticos que usen las vistas: hojas de estilos, *scripts js*, imágenes, etc.

Contendrá las plantillas de las vistas usadas por el motor de plantillas Thymeleaf que usaremos.

Archivo donde se definirán configuraciones propias del proyecto (Ejem.: Puerto usado, uso de plantillas, etc)

Ejercicio 0

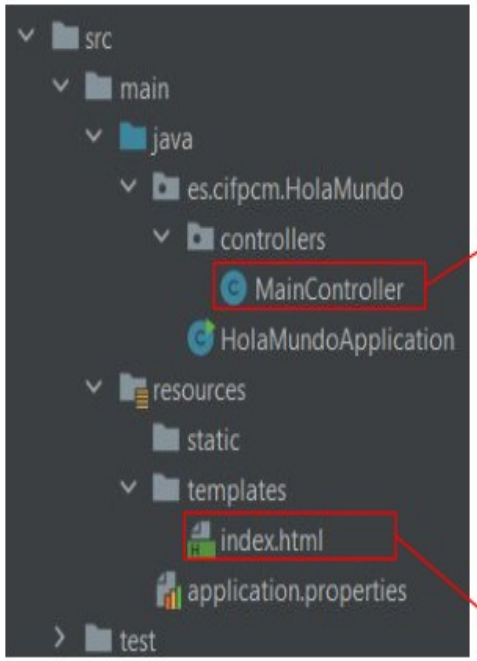
Ejemplo “Hola Mundo”

Para conseguir hacer funcionar nuestro primer ejemplo, es necesario:

- Crear una vista (archivo html) en el directorio “*templates*” que muestre el texto “Hola Mundo”
- Crear un nuevo controlador, en el paquete “controllers” previamente creado, que defina el *endpoint* raíz (“*/*”) de acceso e invoque a la vista creada en el punto anterior.

Tras estos nuevos cambios, la estructura del proyecto será la siguiente:

Spring Web MVC



MainController.java

```java
package es.cifpcm.HolaMundo.controllers;

import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;

@Controller
public class MainController {
    @GetMapping("/")
    public String saluda(){
        return "index";
    }
}
```

index.html

```html
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Hola Mundo</title>
</head>
<body>
    <h1>Hola a todo el Mundo</h1>
</body>
</html>
```

Spring Web MVC

En este punto, tras relanzar el proyecto y acceder al *endpoint* definido en la anotación @GetMapping("/") veremos como se carga la plantilla y su contenido:



Spring Web MVC

Paso de datos del controlador a la vista

Para poder utilizar datos en la vista, que procedan del controlador, hay que realizar las siguientes acciones en los elementos implicados:



- Controlador:
  - Recibir por parámetro un objeto de tipo "Model"
  - Añadir un atributo al "Model" formado por un par clave-valor

```
@GetMapping("/")
public String saluda(Model model){
    model.addAttribute( attributeName: "saludo", attributeValue: "Hola mi gente!");
    return "index";
}
```

- Vista:
  - Añadir atributo (xmlns:th="http://www.thymeleaf.org") en la etiqueta "html" de la plantilla
  - Incluir un atributo "th:text" en el elemento que queramos mostrar el dato

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Hola Mundo</title>
</head>
<body>
    <h1 th:text="${saludo}">Hola a todo el Mundo</h1>
</body>
</html>
```

Spring Web MVC : Controladores

Los controladores se componen principalmente de lo siguiente:

Anotación que transforma una clase simple en un controlador Spring



Anotación para métodos de un controlador que define:

- Tipo de método HTTP
- *Endpoint* de acceso

Anotaciones para otros métodos:

- GetMapping
- PostMapping
- PutMapping
- Etc.

Invocación de la vista.
Se realiza un "return" del nombre de la vista sin extensión.

Función para añadir atributos al modelo que será accesible desde la vista.
Utiliza un sistema clave-valor y pueden incluirse objetos.

Spring Web MVC : Vistas

Las vistas se componen principalmente de lo siguiente:

```
<!DOCTYPE html>
<html lang="en" xmlns:th="http://www.thymeleaf.org">
<head>
    <meta charset="UTF-8">
    <title>Hola Mundo</title>
</head>
<body>
    <h1 th:text="${saludo}">Hola a todo el Mundo</h1>
</body>
</html>
```

Referencia al origen del motor de plantillas utilizado, en este caso Thymeleaf

Nombre del atributo del modelo que queremos acceder

Atributo propio de Thymeleaf para incluir texto en un elemento HTML. Las distintas formas de acceder a los datos desde las plantillas en Spring MVC se detallan en la documentación de Thymeleaf

Ejercicio 1.1

Realiza las siguientes acciones en el proyecto del Ejercicio 0 anterior:

- Crea un nuevo método (también llamado *acción*) en el controlador que se despida de todo el mundo. El nombre de esta acción será “Despedida” y tendrá que cargar una vista distinta de la de saludar.
- Esta acción será ejecutada para el *endpoint* “/despedida”
- El texto para despedirse se pasará en el *Model* desde el controlador que invoca al template.

Spring Web MVC : Vistas Parciales

El uso de vistas parciales nos permite reutilizar elementos que son comunes en todas las páginas (*header*, *footer*, *menu*, *etc*).

En *Thymeleaf* los distintos elementos reutilizables se denominan **fragmentos**.

Existen dos formas de componer las vistas con diversas partes:

- Estilo Jerárquico
  - Consiste en tener un *layout* principal en el que se define un área en el que se cargarán las vistas parciales correspondientes.
  - Este estilo requiere algo más de configuración.
- Estilo inclusivo
  - Se trata de definir algunos elementos como fragmentos e incluirlos en otras vistas.
  - A diferencia de ASP.NET los fragmentos tendrán que estar dentro de una página HTML completa, ya que se ha de definir el atributo "xmlns" que hace referencia a Thymeleaf en la etiqueta <html> para que pueda ser reconocida.
  - Será la manera de trabajar que utilizaremos.

Spring Web MVC : Vistas Parciales

Estilo Inclusivo

Para hacer uso de fragmentos usaremos los siguientes atributos.

- **th:fragment** : Para definir un elemento como fragmento y así poder ser utilizado en otras vistas

```
<nav th:fragment="menu">
  <!--Contenido del Nav-->
</nav>
```

- **th:replace** : El elemento que incluye este atributo se reemplaza por el fragmento referenciado. Se ha de especificar "directorio/archivo::nombre_fragmento"

```
<div th:replace="~{fragments/side::menu}"></div>
```

Ejercicio 1.2

Modifica el proyecto anterior para crear una cabecera común en las dos vistas creadas.

Para ello utilizaremos un fragmento para crear esta cabecera común en ambas páginas (saludar y despedirse).

Incluye dos enlaces en la cabecera para cargar las páginas de “Saluda” y “Despide”

Spring Web MVC

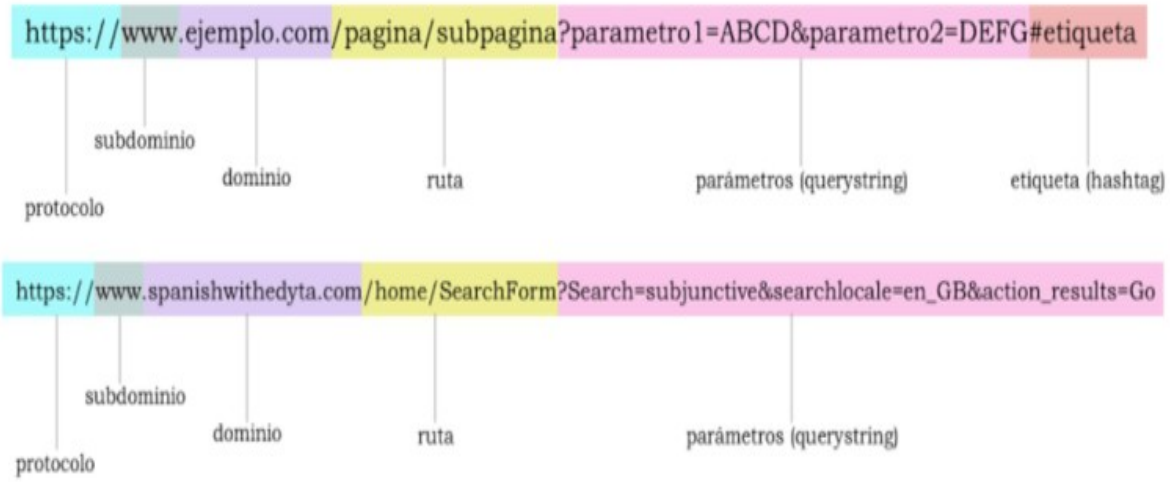
Obtención de Datos de la URL

Los controladores pueden tomar datos de la URL de dos formas distintas:

- Desde un segmento de la ruta (*path*) de la URL
- Mediante la cadena de consulta (*query string*).

Se pueden usar estas estrategias por separado o combinadas.

En los dos casos, será necesario que el método del controlador tenga definidos como parámetros de entrada los valores que desea tomar de la URL, teniendo que coincidir los nombres de los parámetros y de las variables.



Spring Web MVC

Obtención de Datos de la URL: Desde un segmento de la URL

Como ya hemos visto, la URL puede presentar datos necesarios de obtener para poder realizar las acciones necesarias en los métodos de un controlador.

En este caso, el dato puede figurar tanto al final como en el medio de la URL. Ejemplo:

- /usuarios/{userId} → para buscar un determinado usuario por su Id
- /usuarios/{userId}/facturas → para buscar las facturas de un usuario en concreto

Utilizaremos la anotación @PathVariable para acceder a este dato.

La URL podría tener más de un dato a recuperar.

```
@GetMapping("/usuarios/{id}")
public String usuarios(@PathVariable String id, Model model) {
```

Spring Web MVC

Obtención de Datos de la URL: Mediante la cadena de consulta

La cadena de consulta será la parte de la URL que figura detrás del símbolo ?

Se compone por elementos clave-valor

Podrán figurar más de uno de estos pares, siempre concatenados por el símbolo &. Ejemplos:

- http://localhost:8080/?n1=v1
- http://localhost:8080/?n1=v1&n2=v2

Para recuperar estos valores utilizaremos la anotación @RequestParam de la siguiente forma:

```
localhost:8080/?name=Antonio
```

```
@GetMapping("/saludame")
public String sayHello(@RequestParam(value = "name", required=false, defaultValue = "amigo") String name, Model model) {
```

Ejercicio 2

Realizar las siguientes acciones:

- Crea un nuevo método, en tu controlador, llamada “Felicita”
- Este nuevo método ha de recibir los siguientes datos:
  - ID de un usuario (como segmento de la URL)
  - Nombre de un usuario
  - Edad del usuario
- Crea una nueva vista, relacionada con la acción anterior, que reciba los tres datos mediante el Model y felicite al usuario tantas veces como años tenga.
- Todos esas felicitaciones tendrán que ser enlaces que naveguen a la vista de la acción por defecto del controlador “Saludos”

La apariencia de vista creada será la siguiente:

Ejercicio 2


localhost:8080/felicita/3?age=4&name=Pedro
Ejercicio 2
Hoy el usuario con ID: 3 cumple 4 años
Felicidades Pedro
Felicidades Pedro
Felicidades Pedro
Felicidades Pedro

Spring Web MVC: Modelos

En este tema trabajaremos el desarrollo de aplicaciones web MVC con Spring Boot, donde el servidor genera las vistas HTML. En la siguiente unidad abordaremos las aplicaciones REST y SPA.

En la arquitectura MVC que estamos usando, los modelos se corresponden con entidades (tablas) de nuestra futura base de datos.

MVC ofrece la capacidad de pasar objetos de modelos a una vista.

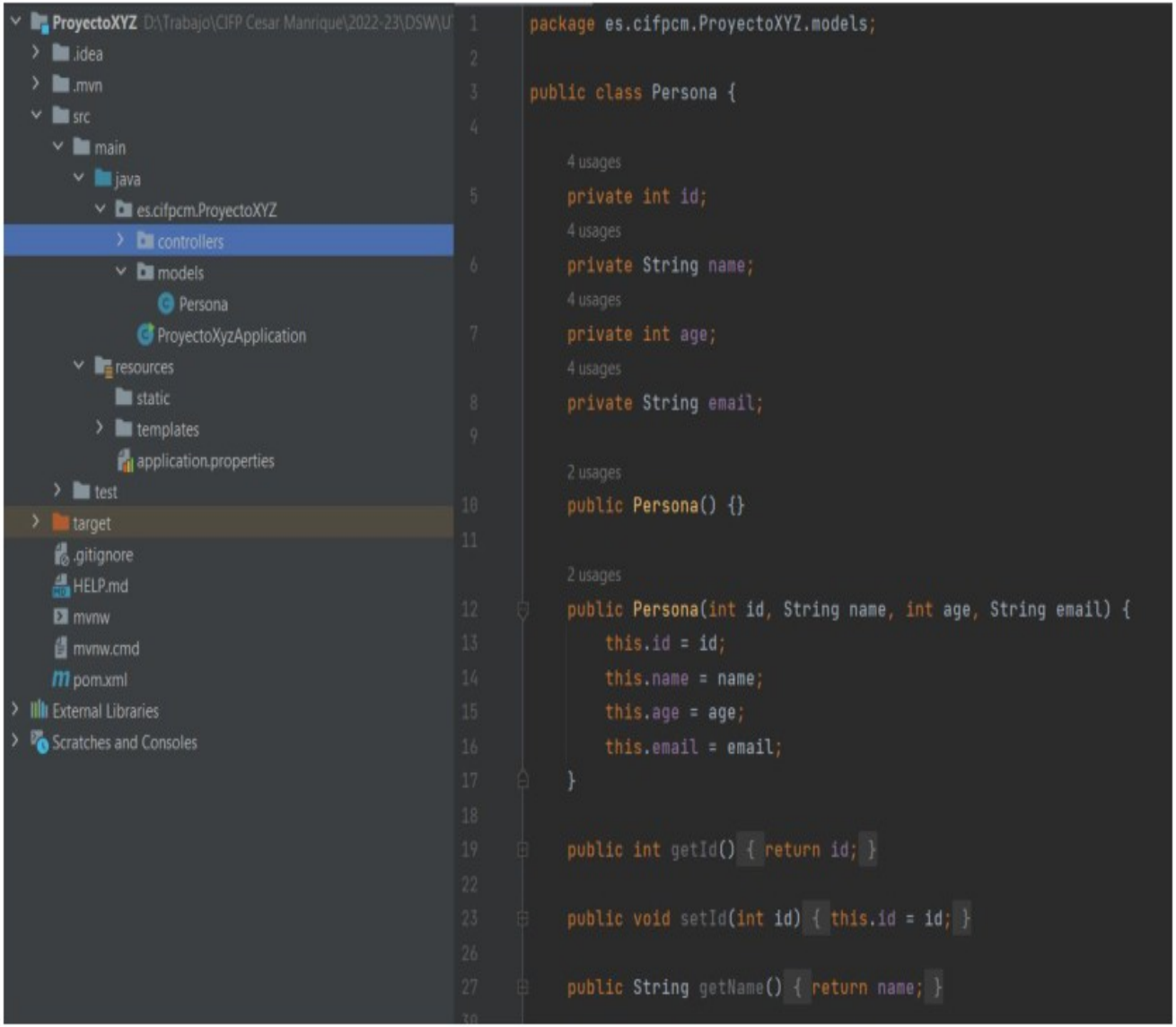
Utilizaremos los modelos como entidades para tomar datos de formularios y para persistir los datos en la BD.

Los modelos serán clases Java como las que ya conocemos (con sus atributos privados, constructores, getters/setters, etc). También las podemos llamar “Entidades”.

A estas clases las dotaremos de anotaciones para definir comportamientos, o para facilitar la validación de estos modelos, como veremos más adelante.

Los modelos los ubicamos en un directorio “Models”.

Spring Web MVC: Modelos



Spring Web MVC: Modelos

Enviar Modelo a la Vista

Para enviar un modelo a la vista, o un conjunto de ellos, utilizaremos la misma técnica antes vista, añadiendo un atributo con la instancia del objeto al elemento de tipo Model.

Creación de un objeto del modelo Persona

```
@GetMapping("/persona")
public String muestraPersona(Model model){

    Persona p1 = new Persona( id: 1,  name: "Antonio",  age: 24,  email: "antonio@gmail.com");
    model.addAttribute( attributeName: "persona", p1);

    return "index";
}
```

Inclusión del objeto de tipo persona en el Model

Modelos: clases Java que representan datos

Una clase Animal representa cada animal de la aplicación.

```
public class Animal {
  private Integer id;
  private String nombre;
  private int vidaMedia;
  private boolean extinto;
  // constructores, getters y setters
}
```

En la siguiente parte veremos cómo persistir estos objetos mediante JPA.

Spring Web MVC: Modelos

Mostrar Campos del Modelo en la Vista

Utilizaremos la misma técnica de ejemplos anteriores.

Para acceder a los atributos del objeto utilizaremos el operador punto (.)

```
<p th:text="${'Id: '+persona.id}"></p>
<p th:text="${'Nombre: '+persona.name}"></p>
<p th:text="${'Edad: '+persona.age}"></p>
<p th:text="${'Correo: '+persona.email}"></p>
```





Servicios e inyección mediante constructor

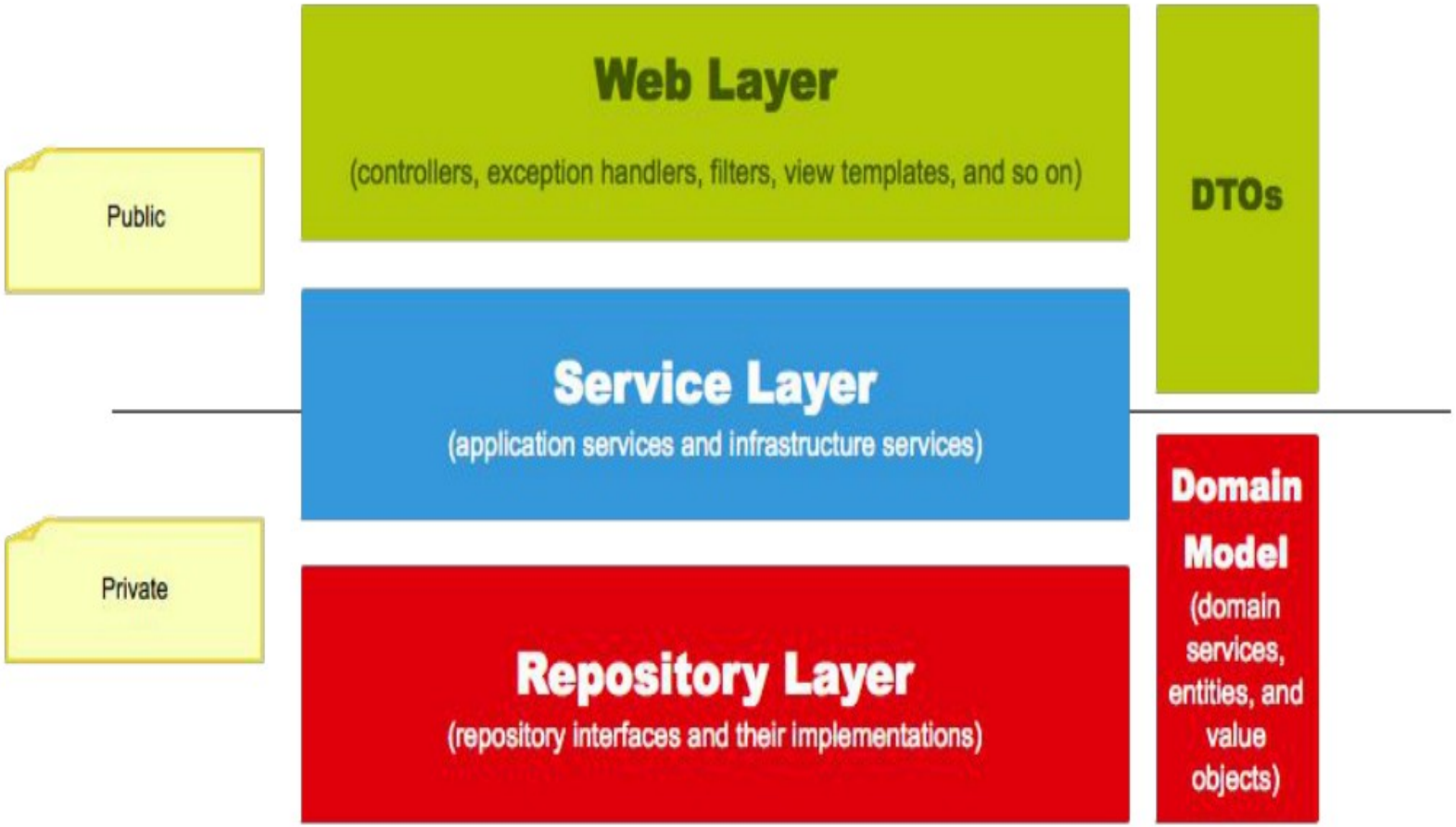
@Service identifica una clase gestionada por Spring. Preferimos la inyección por constructor.

```
@Controller
class AnimalController {
  private final AnimalService service;

  AnimalController(AnimalService service) {
    this.service = service;
  }
}
```

Spring Web MVC: Otros Elementos

Servicios



Ejemplo: modelo, servicio y controlador

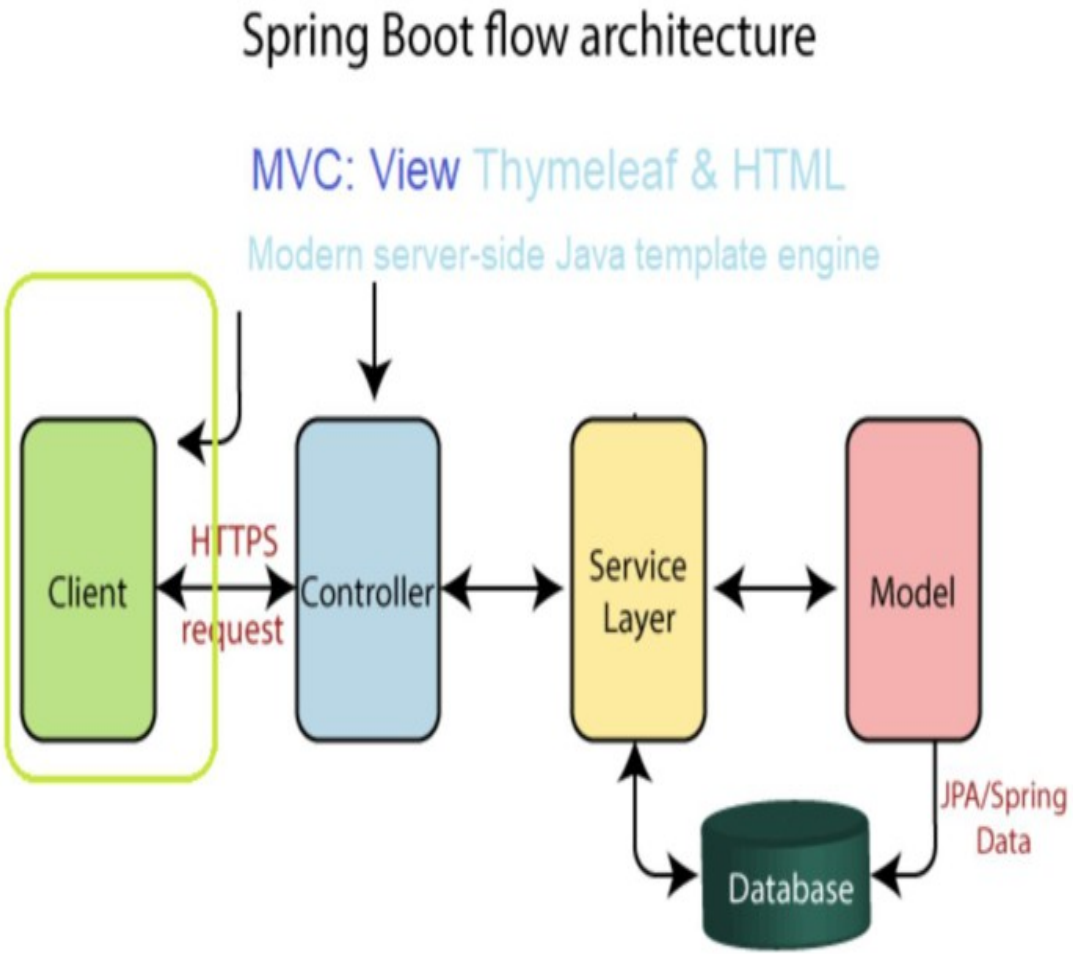
```
class Animal {
  private String nombre;
}
```

```
@Service
class AnimalService {
  List<Animal> findAll()
{ ... }
}
```

```
@Controller
class AnimalController {
  private final AnimalService service;
  AnimalController(AnimalService service) {
    this.service = service;
  }
}
```

El controlador coordina; el servicio resuelve; Animal representa los datos.

Spring Web MVC: Otros Elementos



Spring Web MVC: Modelos

Crear Nuevo Objeto del Modelo

De forma general, y especialmente en las arquitecturas MPA, cuando se pretende crear un nuevo elemento de un determinado tipo de modelo, se sigue el siguiente flujo:

- El cliente comunica al servidor su intención de crear un elemento mediante una petición GET
- El servidor responde a esa petición ofreciendo una página con un formulario que contiene todos los campos necesarios para crear un elemento del modelo.
- El cliente "envía", mediante una petición POST, todo el formulario cumplimentado
- El servidor crea el objeto del modelo solicitado, siempre y cuando todos los campos del formulario superen la validación pertinente.

Este flujo se podría representar de forma gráfica de la siguiente manera:

CREAR UN NUEVO ELEMENTO DE UN MODELO





Petición **GET**: Solicita página con formulario

Respuesta: Página con el formulario del modelo

Petición **POST**: Formulario cumplimentado

Respuesta: Mensaje de OK o Redirección

2 Acciones del Controlador involucradas en el proceso

Spring Web MVC: Modelos

Crear Nuevo Objeto del Modelo

Por tanto, son 2 acciones las involucradas en el proceso de crear un nuevo elemento de un modelo.

- 1ª Acción - GET
  - Su función es la de generar la vista con el formulario completo para crear el modelo
  - Carga una vista que utilizará el modelo para relacionar los elementos label/input con los campos del modelo
- 2ª Acción - POST
  - Recibe la información del formulario en forma de instancia del modelo con sus campos con valores
  - Se encarga de gestionar el modelo recibido → puede persistirlo en la BD o alguna otra función
  - Puede verificar que los datos recibidos sean válidos

Por convenio se utiliza el mismo nombre para estas dos acciones.

Spring Web MVC: Modelos

Crear Nuevo Objeto del Modelo - 1ª Acción

Esta acción se encarga de cargar la vista con el formulario correspondiente.

Se pasará a la vista un objeto vacío de la clase del elemento a crear. De esta forma se emparejarán los campos del formulario con los de la entidad a crear.

```
@GetMapping("/persona/create")
public String crearPersona(Model model){
    model.addAttribute( attributeName: "persona", new Persona());
    return "persona/crear";
}
```

Envío del objeto vacío a la vista mediante un nuevo atributo en el Model utilizando un nombre determinado.

Se carga al vista "crear" del directorio "persona"

Spring Web MVC: Modelos

Crear Nuevo Objeto del Modelo - 1ª Acción

El formulario de la vista ha de definir principalmente:

- La acción a la que se llamará tras enviar el formulario
- Emparejar cada campo del formulario con un campo de la entidad a crear

```
<form action="#"
      th:action="@{/persona/create}"
      th:object="${persona}"
      method="post">

    <label for="name"></label>
    <input type="text" id="name" th:field="*{name}">

    <input type="submit" value="Crear">
</form>
```

Acción a invocar en el envío del formulario

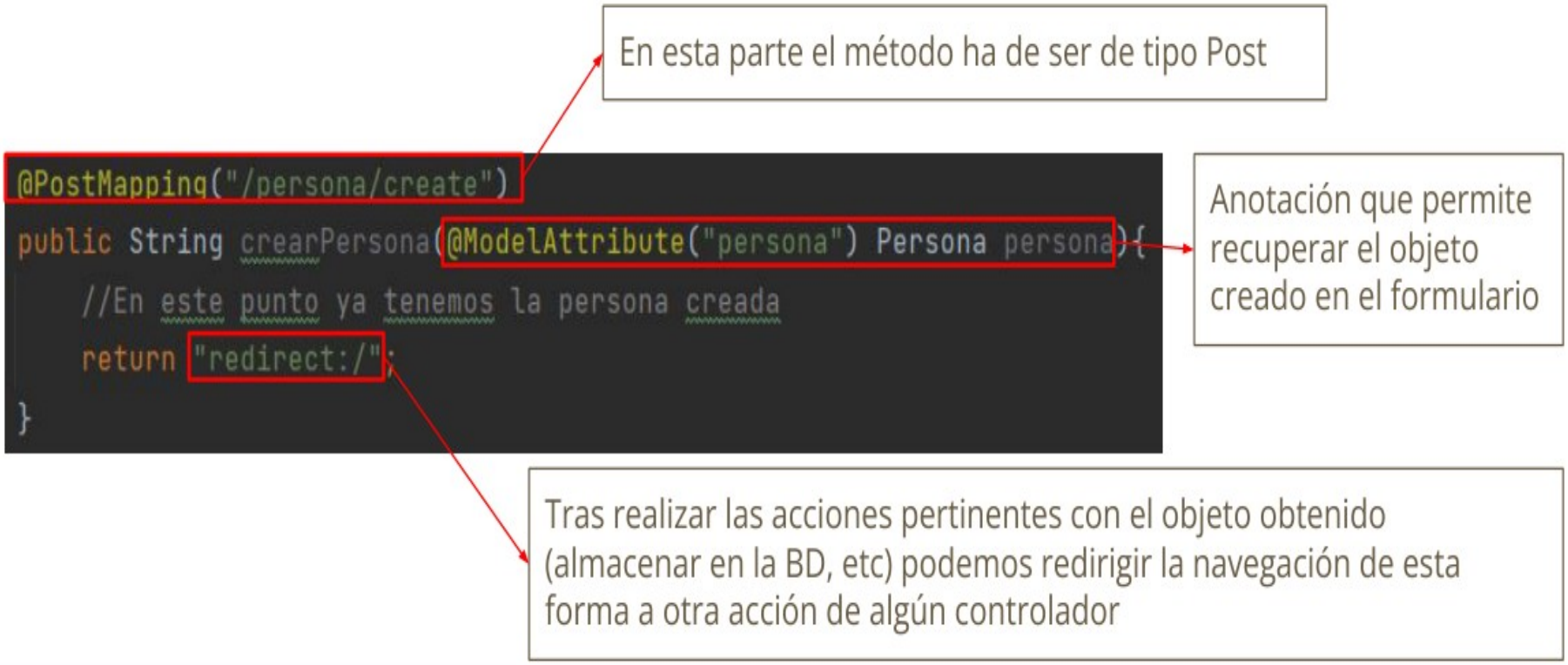
Objeto a crear con el formulario y enviado en el Model

Atributo que empareja este *input* con un campo determinado de la entidad a crear

Spring Web MVC: Modelos

Crear Nuevo Objeto del Modelo - 2ª Acción

Esta acción recibe los datos del formulario en forma de modelo y realiza las acciones pertinentes en función de lo que se quiera conseguir.



Spring Web MVC: Modelos

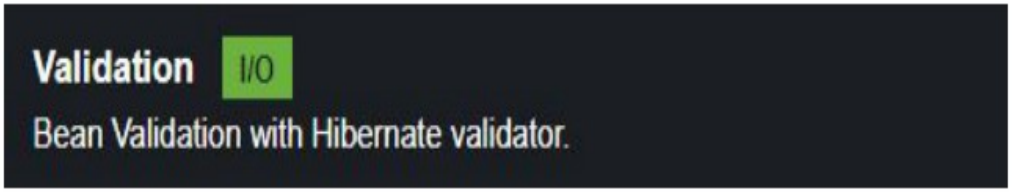
Validación del Modelo en su Creación o Edición

Este proceso involucra de diferente forma a varios archivos del proyecto que participan en la creación/modificación de un elemento de un modelo.

- Clase que define el modelo
- Controlador llamado al hacer *submit* del formulario
- Vista del formulario para crear el modelo

Cada uno de estos elementos tiene una labor determinante en la validación del modelo para su creación o modificación.

Para aplicar validación tendremos que incluir una nueva dependencia al POM:



Spring Web MVC: Modelos

Validación del Modelo en su Creación o Edición - Modelo

Como se comentó anteriormente, los campos de un modelo pueden ir acompañados de anotaciones de validación que definen restricciones a validar sobre cada uno de ellos. Pueden definir un mensaje de error personalizado.

Algunos ejemplos y uso de estas puede ser:

- @NotNull
- @NotEmpty
- @NotBlank
- @Min y @Max
- @Size
- @Pattern
- @Email
- y hay más...

```
@NotBlank(message = "Name is mandatory")
private String name;

@NotBlank(message = "Email is mandatory")
private String email;
```

```
@NotBlank
@Size(min = 3, max = 12)
private String username;

@NotBlank
@Size(min = 6)
private String password;
```

Spring Web MVC: Modelos

Validación del Modelo en su Creación o Edición - Controlador

La validación de datos recibidos en el servidor es una acción imprescindible, ya que es posible saltarse las validaciones del lado cliente.

En la acción (de tipo POST) que recibe el modelo con los datos del formulario se realizará la siguiente comprobacion:

- Que los datos que trae el modelo generado con los datos del formulario cumple con todas las restricciones definidas en el modelo mediante los atributos de validación

Veamos un ejemplo:

Spring Web MVC: Modelos

Validación del Modelo en su Creación o Edición - Controlador

```
@PostMapping("/persona/create")
public String crearPersona(@Valid @ModelAttribute("persona") Persona persona,
                           BindingResult bindingResult){
    if(bindingResult.hasErrors()){
        //Modelo inválido
        return "persona/crear";
    }else{
        //Modelo válido
        return "redirect:<alguna_parte>";
    }
}
```

Validación: del modelo al formulario

1 · Modelo

@NotBlank

@Size(min = 3, max = 15)

private String nombre;

2 · Controlador

@Valid @ModelAttribute

BindingResult result

Si hay errores, devolver

la vista del formulario.

3 · Vista

th:field="*{nombre}"

th:errors="*{nombre}"

Mostrar al usuario los

errores del servidor.

Nunca depender exclusivamente de la validación HTML del navegador.

Spring Web MVC: Modelos

Validación del Modelo en su Creación o Edición - Formulario

Se adaptará el formulario para poder mostrar los posibles errores que fueran reportados por el controlador al validar el modelo recibido.

Utilizaremos atributos de Thymeleaf para modificar la apariencia del formulario para conseguir dos cosas:

- Añadir/quitar/omitir clases que definen estilos CSS en función de condiciones:
  - th:classappend="${#fields.hasErrors('<fieldName>')} ? '<clase1>' : '<clase2>'"
  - Se puede usar ese operador ternario para evaluar valores de algún campo del modelo y añadir/quitar/omitir otros elementos de la vista, como los atributos. Se usaría el atributo *th:attrappend="....."*
- Mostrar determinados elementos de la vista para mostrar mensajes de error en caso de que exista un error con un determinado campo, usando dos atributos:
  - th:if="${#fields.hasErrors('<fieldName>')}" th:errors="*{<fieldName>}"

Spring Web MVC: Navegación

En una aplicación **Spring Boot MVC**, la navegación entre páginas se basa en **peticiones HTTP tradicionales**, donde **cada cambio de vista implica una nueva solicitud al servidor**.

Es decir:

- El **navegador (cliente)** realiza una **petición HTTP (GET o POST)** a una ruta del servidor.
- El **controlador** de Spring Boot procesa esa petición, ejecuta la lógica necesaria (servicios, modelos, etc.) y **devuelve una vista** (una plantilla HTML renderizada con datos).
- Spring Boot **renderiza la vista usando Thymeleaf** y la envía al navegador, reemplazando el contenido anterior.

Cada vez que cambiamos de página, el navegador **recarga completamente la vista HTML**.

Spring Web MVC: Navegación

Flujo de navegación más común

1. El usuario hace clic en un enlace o envía un formulario.
2. El navegador realiza una petición HTTP a una URL concreta (por ejemplo, `/alumnos`, `/nuevo`, `/guardar`).
3. Spring Boot busca el controlador con la ruta correspondiente (`@GetMapping` o `@PostMapping`).
4. El controlador ejecuta el código necesario y devuelve el nombre de la vista o realiza una redirección a otra URL de la web.
5. Spring Boot renderiza la plantilla ubicada en `src/main/resources/templates/`.
6. El navegador muestra la nueva página.

Spring Web MVC: Navegación

VISTAS: Enlaces y Rutas

Thymeleaf usa el atributo `th:href` para generar enlaces dinámicos según las rutas del controlador.

```
<a th:href="@{/alumnos}">Listado</a>
<a th:href="@{/alumnos/nuevo}">Nuevo alumno</a>
<a th:href="@{/alumnos/detalle/{id}(id=${alumno.id})}">Ver detalle</a>
```

Rutas estáticas

Rutas dinámica que incluye un valor en un segmento del Path

```
<a th:href="@{/buscar(pagina=${paginaActual}, filtro=${filtro}, orden='asc')}">
```

Rutas dinámica que incluye variables en QueryString

```
/buscar?pagina=2&filtro=activos&orden=asc
```

Spring Web MVC: Navegación

VISTAS: Enlaces y Rutas

RECUERDA:

- @{...} → indica a Thymeleaf que estás generando una **URL dinámica**.
- Entre paréntesis (...) puedes pasar los **parámetros de la query string**.
- Cada parámetro se define como clave=${valor} si se quiere usar un valor contenido en el Model del controlador, o como clave='valor' si se quiere dar un valor estático.
- Si añades varios, se separan con comas.
- **No** hay que escribir URLs absolutas usando el atributo `href` de html, ni aunque sea una ruta estática.
- Los formularios, al enviarlos, producen una navegación a la ruta guardada en el atributo `th:action`

```
<form th:action="@{/alumnos/guardar}" method="post">
    <input type="text" name="nombre" placeholder="Nombre">
    <button type="submit">Guardar</button>
</form>
```

Spring Web MVC: Navegación

CONTROLADORES: definición de rutas y redireccionamiento

Las acciones del controlador definen las rutas de acceso mediante `@GetMapping` o `@PostMapping`

Un controlador puede ir decorado con la anotación `@RequestMapping("/<ruta>")` haciendo que eso sea el prefijo de todas las rutas de sus acciones.

Las acciones del controlador, tras finalizar su trabajo, pueden realizar dos cosas:

- Cargar una vista, retornando la ruta y nombre donde se encuentra el template de la vista

```
return "alumnos/lista";
```

- Redirigir a otra acción retornando `"redirect:<ruta>"`

```
return "redirect:/alumnos";
```

Spring Web MVC: Navegación

EJEMPLO

templates/alumnos/lista.html

```html
<h1 th:text="${titulo}"></h1>
<a th:href="@{/alumnos/nuevo}">Añadir nuevo alumno</a>
```

templates/alumnos/formulario.html

```html
<h1 th:text="${titulo}"></h1>
<form th:action="@{/alumnos/guardar}" method="post">
    <input type="text" name="nombre" placeholder="Nombre">
    <button type="submit">Guardar</button>
</form>
<a th:href="@{/alumnos}">Volver al listado</a>
```

```java
@Controller
@RequestMapping("/alumnos")
public class AlumnoController {

    @GetMapping
    public String listar(Model model) {
        model.addAttribute("titulo", "Listado de Alumnos");
        return "alumnos/lista"; // -> templates/alumnos/lista.html
    }

    @GetMapping("/nuevo")
    public String mostrarFormulario(Model model) {
        model.addAttribute("titulo", "Nuevo Alumno");
        return "alumnos/formulario"; // -> templates/alumnos/formulario.html
    }

    @PostMapping("/guardar")
    public String guardar(Alumno alumno) {
        // Guardar datos del alumno...
        return "redirect:/alumnos"; // redirige a la lista
    }
}
```

Después de guardar: Post / Redirect / Get

Después de una operación POST correcta, redirigimos a una URL de lectura.

POST

Envía formulario

REDIRECT

Respuesta 302

GET

Muestra el listado

```
return "redirect:/animals";
```

Evita que actualizar la página reenvíe el formulario.

OPERACIONES CRUD en Spring Boot

El término **CRUD** proviene de las siglas en inglés de las cuatro operaciones básicas que se realizan sobre los modelos en cualquier aplicación:

Letra	Operación	Significado	Acción en la aplicación
C	*Create*	Crear	Insertar un nuevo registro en el modelo (nuevo elemento).
R	*Read*	Leer	Consultar datos existentes (listar o mostrar detalles).
U	*Update*	Actualizar	Modificar datos de un elemento existente.
D	*Delete*	Eliminar	Borrar un elemento existente.

CRUD: verbos HTTP recomendados

Consultar	GET: listado y detalle
Crear	GET: formulario · POST: guardar
Editar	GET: formulario · POST: actualizar
Eliminar	GET: confirmación · POST: eliminar

Nunca ejecutar un borrado al visitar un enlace GET.

OPERACIONES CRUD en Spring Boot

R — Read (Leer o consultar datos)

Permitir al usuario **consultar los datos existentes** del modelo, bien sea de forma general (listado) o específica (detalle).

a) Lectura de todos los elementos (Listar)

1. **Definir una ruta** que recupere todos los registros del modelo desde el servicio.
2. Enviar esa lista al **modelo de la vista** para que pueda mostrarse en una tabla o listado.
3. **Diseñar una vista** que muestre cada elemento y, si procede, ofrezca enlaces para ver detalles, editar o eliminar.
4. Asegurarse de que el listado se actualiza después de cada operación de creación, edición o eliminación.

b) Lectura de un solo elemento (Detalle)

1. **Definir una ruta** que reciba un identificador (por ejemplo, el ID del elemento). Una vista, mediante th:href incluye un ID en la ruta.
2. El **controlador solicita al servicio** los datos del elemento concreto.
3. Enviar el objeto recuperado al modelo de la vista.
4. **Diseñar una vista de detalle** que muestre la información completa del elemento.
5. Incluir enlaces para volver al listado o realizar acciones (editar, eliminar...).

OPERACIONES CRUD en Spring Boot

C — Create (Crear nuevos elementos)

Objetivo:

Permitir al usuario **añadir un nuevo registro** del modelo (por ejemplo, un nuevo alumno, producto o cliente).

Pautas:

1. **Definir una ruta** que muestre un formulario vacío al usuario para introducir los datos del nuevo elemento.
   (Petición GET → muestra la vista del formulario).
2. **Diseñar un formulario HTML** con los campos correspondientes al modelo.
3. Al enviar el formulario, el navegador realiza una **petición POST** con los datos introducidos.
4. El **controlador recibe los datos**, los valida (por ejemplo, que no haya campos vacíos o inválidos).
5. Si los datos son correctos, los **envía al servicio** para que los guarde (en memoria o base de datos).
6. Tras guardar, **redirigir al listado** general para mostrar el nuevo elemento junto a los demás.
   (Se aplica el patrón *Post/Redirect/Get* para evitar reenvíos del formulario).

OPERACIONES CRUD en Spring Boot

U — Update (Actualizar elementos existentes)

Objetivo:

Permitir al usuario **modificar los datos** de un elemento que ya existe.

Pautas:

1. **Definir una ruta** que cargue los datos del elemento a editar, identificándolo por su ID.
   (Petición GET → muestra el formulario con los campos ya rellenos). Se ha de recuperar el modelo concreto del ID recibido en la petición GET para poder rellenar el formulario.
2. Mostrar un **formulario HTML similar al de creación**, pero con los valores actuales del elemento.
3. Al guardar los cambios, el formulario realiza una **petición POST** al servidor.
4. El **controlador valida los nuevos datos** y llama al servicio para actualizar el registro existente.
5. Tras la actualización, **redirigir nuevamente al listado o a la página de detalle** para mostrar la información actualizada.

OPERACIONES CRUD en Spring Boot

D — Delete (Eliminar elementos)

Objetivo:

Permitir al usuario **borrar un elemento existente** del modelo.

Pautas:

1. **Definir una ruta** que reciba el ID del elemento que se desea eliminar.
2. El **controlador llama al servicio** para ejecutar la eliminación.
3. Tras eliminar, **redirigir al listado principal**, actualizando la información visible.
4. Añadir una **confirmación previa** en la vista (por ejemplo, una alerta o ventana de confirmación) para evitar eliminaciones accidentales.

Ejercicio 3

Generar un proyecto Spring MVC que permita realizar las siguientes acciones sobre elementos de un modelo que representa a Animales:

- Listar el conjunto de animales
- Mostrar detalles de un animal por su id
- Crear nuevos animales
- Actualizar un animal por su id
- Eliminar un animal por su Id

El Id de un animal será algo que tenga que gestionar la aplicación y por tanto no tendrá que especificarlo el usuario en ningún caso.

Ejercicio 3

El modelo que representa a estos animales tendrá que tener los siguientes atributos (con sus respectivas restricciones)

- Id
- Nombre (campo obligatorio, min. 3 caracteres y máx. 15)
- VidaMedia (campo obligatorio, numérico, valor mínimo 0 y máximo valor 600)
- Extinto (tipo de dato lógico y obligatorio)

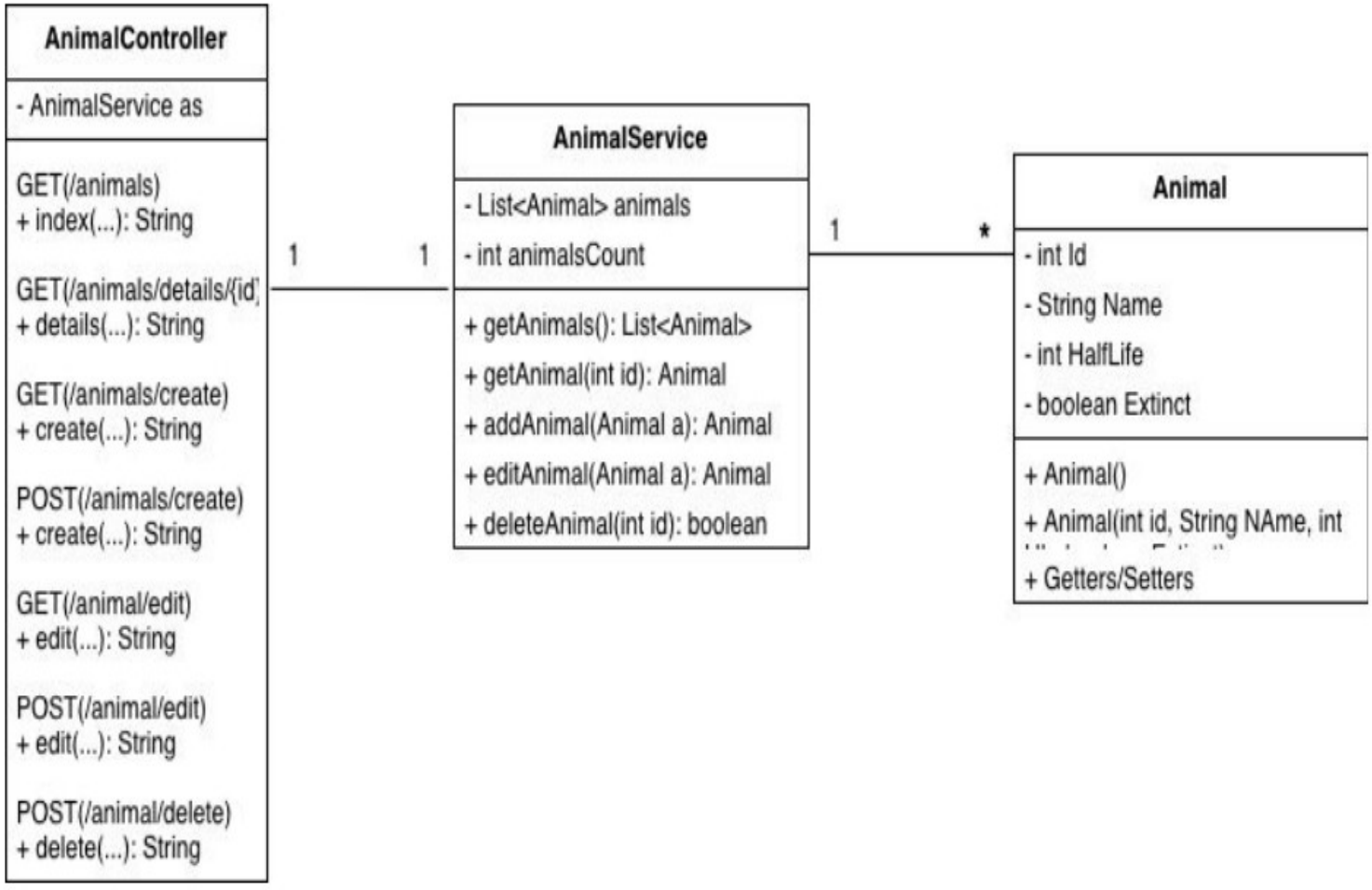
La creación de nuevos animales requiere de validación de los campos en el cliente, adaptando el formulario a los requisitos, y en el servidor.

Todos los textos de la página tendrán que aparecer en castellano (nombres y validaciones).

Se tendrá que usar *Bean* de Servicio para gestionar la simulación de persistencia de los datos, como hemos visto en clase.

Ejercicio 3

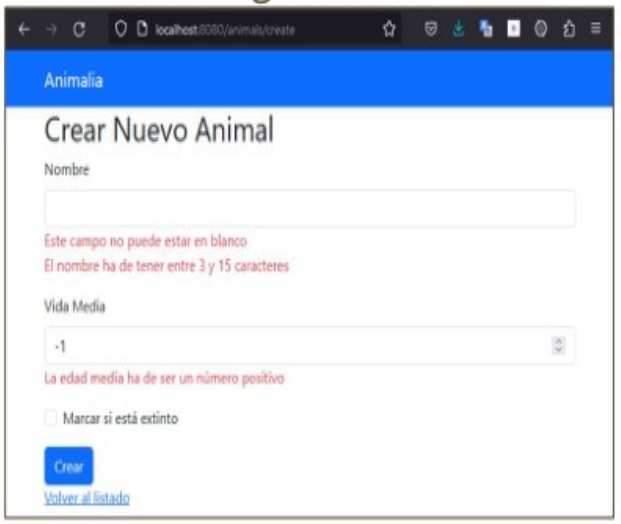
Las clases desarrolladas se han de corresponder con el siguiente diagrama:



Ejercicio 3

Las vistas generadas han de tener un aspecto similar a las siguientes:







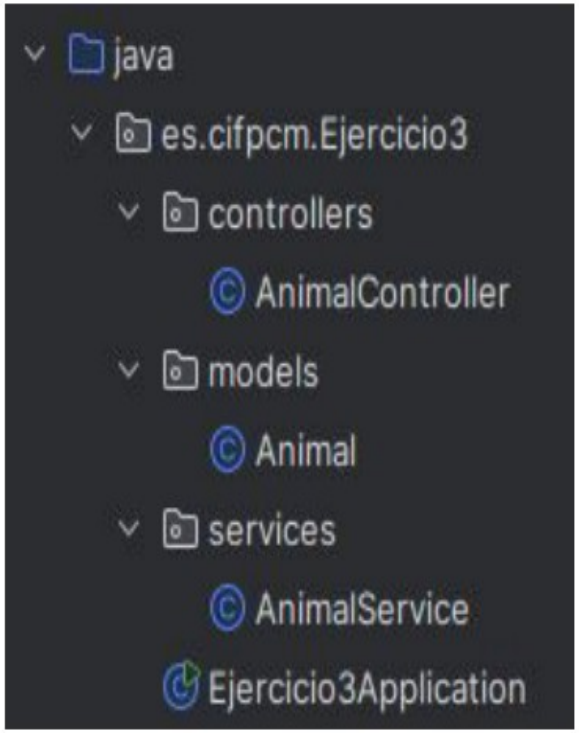




Ejercicio 3

PASO 1:

Generar la estructura del proyecto correctamente.



Ejercicio 3

PASO 2:

Crear la clase AnimalService siguiendo las siguientes directrices:

- La clase ha de llevar la anotación @Service
- La clase ha de tener como atributo una lista de elementos de tipo Animal.
- Se ha de añadir un elemento de tipo Animal en la lista en su creación de la siguiente manera:

```
private List<Animal> animales = new ArrayList<~>(Arrays.asList(  8 usages
        new Animal( id: 1,  nombre: "Dodo",  vidaMedia: 50,  extinto: true)
));
```

- Se ha de crear un método que retorne la lista de animales, que se puede llamar *GetAnimals*

Ejercicio 3: conectar el controlador con el servicio

```
@Controller
@RequestMapping("/animals")
public class AnimalController {
  private final AnimalService service;
  public AnimalController(AnimalService service) {
    this.service = service;
  }
  @GetMapping
  String listar(Model model) {
    model.addAttribute("animals", service.findAll());
    return "animals/index";
  }
}
```

Thymeleaf: recorrer una lista

El controlador añade «animales» al Model y la plantilla recorre sus elementos.

```
model.addAttribute("animals", service.findAll());
```

```
<ul>
  <li th:each="animal : ${animals}"
      th:text="${animal.nombre}"></li>
</ul>
```

Resultado: una línea <li> por cada animal.

Ejercicio 3

PASO 4:

Crear la vista Animals, que se encargara de mostrar los elementos de la lista de animales que contiene el Model.

Utilizar el elemento th:each de Thymeleaf para iterar sobre la lista y pintar sus elementos.

Con esto habríamos creado la primera funcionalidad de Listar de nuestra aplicación.

Ejercicio 3

PASOS SIGUIENTES:

1. Mostrar Detalles de un animal

```
<a class="btn btn-outline-success" th:href="@{/animals/details/{id}(id=${animal.Id})}">Detalles</a>
```

2. Crear Animal
3. Editar Animal
4. Borrar Animal